

MCP Runtime Contract

The **Multi-Call Provider (MCP)** integration layer allows the RAGenius Execution Subsystem to dynamically discover and call tools exposed by external providers (such as Notion, Asana, or custom connectors). This document defines how MCP providers are configured, how tools are discovered and mapped into the internal schema, and how MCP tools are executed at runtime.

Provider Configuration

Each MCP provider must be registered with the following information:

Field	Type	Description
<code>id</code>	UUID	Unique identifier.
<code>name</code>	string	Name of the provider (e.g. <code>notion</code>).
<code>server_url</code>	string	Base URL of the MCP server.
<code>auth_type</code>	string	<code>none</code> , <code>bearer</code> , <code>oauth2</code> , or <code>mcp_session</code> .
<code>auth_config</code>	object	Provider-specific authentication configuration (e.g. client ID, scopes, token endpoint).
<code>enabled</code>	boolean	Whether the provider is active.
<code>discovered_tools</code>	array	Cached list of tools discovered from this provider.
<code>last_discovered_at</code>	timestamp	Last time discovery was run.

Providers may specify additional configuration, such as rate limits or default timeouts, in the `auth_config` field. Provider configuration should be stored in a secure database table (see `persistence-schema-contract.md`).

Tool Discovery

1. **Discovery endpoint:** MCP servers expose an endpoint (e.g. `/discover`) that returns metadata about available tools. The execution subsystem should call this endpoint on a schedule or on demand via `POST /v1/tools/discover/mcp`.
2. **Authentication:** The MCP client must authenticate with the provider using the configured `auth_type`. For `bearer`, a token should be loaded from environment variables or a secrets manager. For `oauth2`, the client should obtain a token via the OAuth flow and cache it. For `mcp_session`, the provider might issue session keys.

3. **Tool mapping:** Each discovered tool must be converted into an internal `ToolDefinition` with fields:
4. `id`: Prefix with the provider name to ensure uniqueness (e.g. `mcp.notion.create_page`).
5. `name`: Provided by the MCP.
6. `providerType`: Set to `mcp`.
7. `inputSchema` and `outputSchema`: Map the provider's request/response models into JSON Schemas. If the provider exposes these in OpenAPI format, convert accordingly.
8. `permissionScopes`: Assign appropriate scopes based on the tool's side effects (e.g. `external_api.read`, `external_api.write`). Unknown tools should default to `require_confirmation` until policies are defined.
9. `timeoutMs` and `sideEffecting`: Copy or set reasonable defaults.
10. **Registration:** Once mapped, the tool should be inserted into the local `tools` table (with `provider_type='mcp'`) and made available for skills.
11. **Caching:** Store the list of discovered tools in `discovered_tools` along with `last_discovered_at`. Subsequent discovery runs can fetch updates incrementally. Tools removed from the provider should be disabled locally until explicitly deleted.

Tool Execution

When a skill calls an MCP tool, the tool engine follows these steps:

1. **Retrieve tool definition:** Look up the tool by ID in the local registry. If not found or disabled, reject with a `validation` error.
2. **Validate input:** Validate the input against the mapped `inputSchema`. Reject mismatches with a `validation` error.
3. **Check permission:** Enforce permission policies for the tool's `permissionScopes` as described in `auth-and-permission-policy.md`. Tools discovered via MCP often have side effects (e.g. creating or deleting objects) and may require confirmation.
4. **Prepare request:** Construct the MCP request by mapping input fields into the provider's expected payload. The provider's API path and method should be part of the tool metadata.
5. **Authenticate:** Use the provider's auth mechanism. Inject bearer tokens or session keys as needed. Do not log tokens.
6. **Call provider:** Send the HTTP request to the provider's endpoint. Respect the configured timeout and handle network errors. Apply retries if configured.
7. **Handle response:** Convert the provider's response into the shape defined by `outputSchema`. Map fields into the execution context via the step's `output_mapping`.
8. **Normalize and log:** Store a summary of the request and response, redacting any tokens or sensitive fields. Classify any errors returned by the provider as `tool` or `external_api` errors, and provide a suggested action.

Error Handling

MCP calls can fail in several ways:

- **Authentication error:** The provider rejects the token or credentials. Classify as `external_api` error. Suggested action: refresh credentials or update auth configuration.
- **Validation error:** The provider returns 400 due to invalid input mapping. Classify as `tool` error. Suggested action: adjust skill's input or workflow mapping.
- **Rate limit exceeded:** The provider returns 429. Classify as `external_api` error. Suggested action: throttle calls or implement backoff.
- **Server error:** Provider returns 5xx. Classify as `external_api` error. Suggested action: retry after delay or contact provider support.
- **Unknown tool:** If the provider returns an error that the tool does not exist, disable the tool locally and inform the administrator.

Security Considerations

1. **Least privilege:** Assign the minimum scopes necessary for each MCP tool. Most creation or mutation tools should be restricted or require confirmation. Read-only tools can be auto-allowed.
2. **Secrets management:** Store provider tokens and client secrets in environment variables or a secure secrets manager. Do not commit them to source control or logs.
3. **Timeouts:** Set sensible timeouts (e.g. 30 seconds) for MCP calls to avoid hanging executions.
4. **Redaction:** Redact bearer tokens, session keys, and sensitive response data from logs.
5. **Discovery frequency:** Limit discovery calls to a reasonable interval (e.g. once per hour) to reduce load on providers. Provide a manual override to refresh tools on demand.

Suggested Confirmation Flow

Because MCP tools often mutate external systems, we recommend defaulting to `require_confirmation` for write operations. When confirmation is triggered, include details like the tool name, provider, and a summary of the input (redacted if necessary) so that the user can make an informed decision.
